

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Bases de Datos II

Primer Proyecto: API Restaurantes, Documentación

Cesar Fabricio Herrera Rodríguez

Catherin Madriz Badilla

Kenneth Obando Rodríguez

07/05/2026

Diagrama lógico	1
Diagrama físico	3
Microservicios y Responsabilidades.....	4
API Principal	5
Microservicio de Búsqueda	5
Servicio de Base de Datos MongoDB.....	6
Servicio de Base de Datos PostgreSQL	7
Keycloak	7
Redis.....	7
Balanceador de Carga (Nginx)	8
Pipeline CI/CD	8
Flujo de Datos y Uso del Balanceador	8

Diagrama lógico

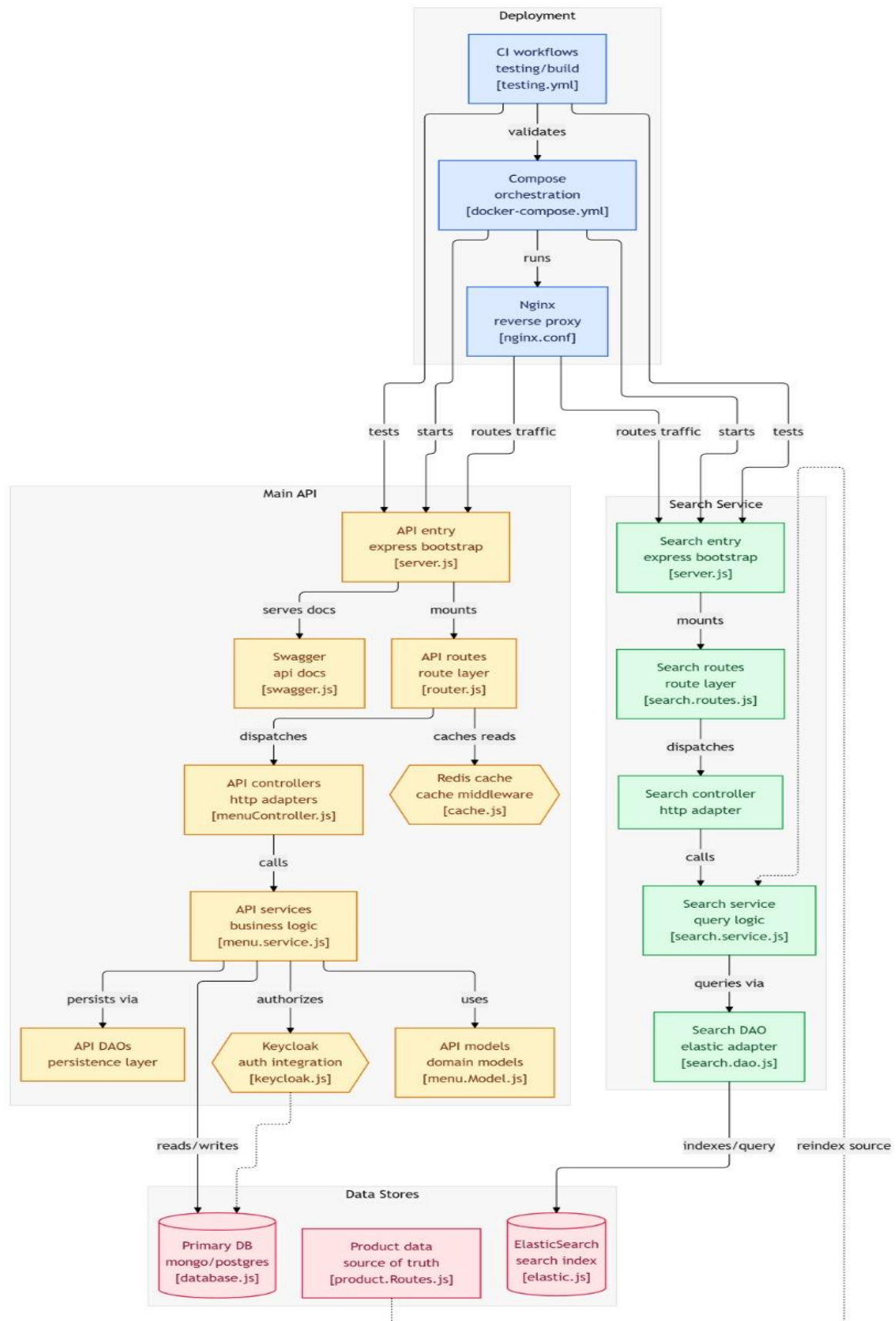
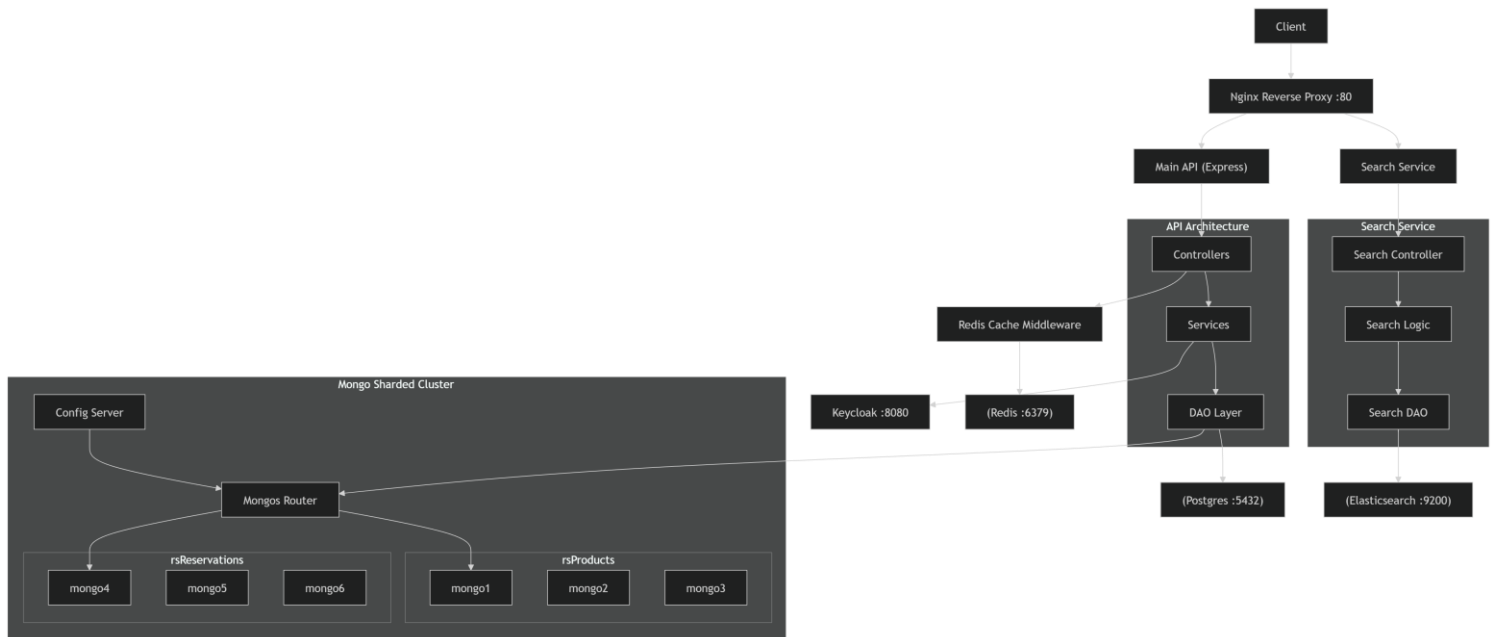


Diagrama físico



Microservicios y Responsabilidades

API Principal

La API principal es el microservicio encargado de gestionar toda la lógica de negocio del sistema de restaurantes. Este servicio expone endpoints REST para administrar menús, restaurantes, usuarios, reservas y órdenes.

Además, centraliza las validaciones del sistema y coordina la comunicación con la base de datos, Redis y el microservicio de búsqueda.

La API fue desarrollada utilizando una arquitectura por capas basada en routes, controllers, services y DAOs, permitiendo separar la lógica de negocio, el acceso a datos y el manejo de solicitudes HTTP.

La autenticación y autorización de usuarios se realiza mediante Keycloak utilizando tokens JWT.

Responsabilidades

- Gestionar menús, restaurantes, reservas, ordenes, usuarios
- Validar reglas de negocio.
- Exponer endpoints REST.
- Autenticar y autorizar usuarios mediante Keycloak.
- Acceder a PostgreSQL o MongoDB mediante el patrón DAO
- Utilizar Redis para cachear respuestas frecuentes.
- Comunicarse con el microservicio de búsqueda.
- Manejar errores y validaciones del sistema.

Microservicio de Búsqueda

El microservicio de búsqueda es el encargado de gestionar las búsquedas de productos utilizando Elasticsearch como motor de indexación y consulta.

Este servicio funciona de forma independiente de la API principal y permite realizar búsquedas textuales rápidas, filtrado por categorías y reindexación manual de productos.

La arquitectura del microservicio se encuentra separada en capas controllers, services y Daos

Responsabilidades:

- Realizar búsquedas de productos
- Filtrar productos por su categoría
- Gestionar índices en Elasticsearch
- Reindexar productos manualmente
- Procesar consultas optimizadas sobre productos indexados

Servicio de Base de Datos MongoDB

El servicio de MongoDB es el componente encargado de la persistencia de datos utilizando una arquitectura distribuida basada en replicación y sharding.

En la arquitectura del proyecto se configuraron shards para las colecciones de reservas y menús, donde cada shard cuenta con su propio conjunto de réplicas compuesto por un nodo primario y dos nodos secundarios.

Responsabilidades

- Persistir información del sistema.
- Garantizar alta disponibilidad mediante replicación.
- Distribuir datos entre múltiples nodos.
- Mantener redundancia y tolerancia a fallos.

Configuración Implementada

Replica Sets

Cada shard fue configurado con:

- 1 nodo primario.
- 2 nodos secundarios.

Sharding

Se implementó particionamiento de datos para:

- Reservas.
- Menús con sus productos.

Servicio de Base de Datos PostgreSQL

El servicio de PostgreSQL es el componente encargado de la persistencia de datos relacional dentro del sistema.

Esta base de datos se utiliza como alternativa a MongoDB y puede ser seleccionada dinámicamente mediante configuración, sin necesidad de modificar la lógica de negocio de la aplicación.

La integración fue desarrollada utilizando el patrón DAO, permitiendo desacoplar el acceso a datos de los servicios y controladores de la API principal.

Responsabilidades

- Gestionar transacciones y consistencia de datos.
- Ejecutar operaciones CRUD.
- Garantizar integridad referencial.
- Servir como alternativa configurable a MongoDB.
- Almacenar información de productos, usuarios, menús, restaurantes, reservas y órdenes.

Keycloak

Keycloak es el microservicio encargado de realizar todo el manejo y autenticación de los usuarios. Todas las rutas están protegidas por su autenticador, así como también este mismo provee un token JWT para realizar diferentes tipos de consultas HTTP

Responsabilidades:

- Autenticar a todos los usuarios
- Manejo de los datos de los usuarios, sus contraseñas y demás identificadores
- Protección de rutas, funcionando como middleware

Redis

Redis actúa como un servicio de cache para las solicitudes GET, guardando la información obtenida en estas solicitudes, haciendo que cada vez que sean llamadas, primero se busque si estas residen en Redis, en el caso de que no lo sean, almacena la respuesta utilizando el patrón CACHE-ASIDE y da un tiempo de expiración de 2 minutos a la respuesta, siguiendo estas mismas políticas de expiración para todas las rutas GET

Responsabilidades:

- Recibir solicitudes del cliente
- Actúa como middleware entre el controlador y los DAOs
- Disminuye considerablemente el tiempo de respuesta de las solicitudes
- Anular el cacheo de solicitudes que no sean GET, para evitar problemas entre solicitudes con misma ruta, pero diferente método

Balanceador de Carga (Nginx)

El balanceador de carga es el encargado de recibir todas las solicitudes que entran al sistema y redirigirlas hacia el microservicio correspondiente.

Se utilizó Nginx como balanceador del sistema para poder distribuir entre múltiples instancias de los servicios, además el balanceador centraliza el acceso a la API y al microservicio de búsqueda mediante rutas específicas.

Responsabilidades

- Recibir solicitudes del cliente
- Redirigir tráfico hacia los microservicios correspondientes.

- Distribuir carga entre múltiples instancias de la API principal.
- Distribuir carga entre múltiples instancias del microservicio de búsqueda.
- Facilitar la escalabilidad horizontal mediante Docker Compose.

Pipeline CI/CD

Un flujo CI/CD es un proceso automatizado que se encarga de validar, construir y preparar una aplicación cada vez que se realizan cambios en el código. Su objetivo es asegurar que el sistema se mantenga estable, funcional y listo para ser entregado en cualquier momento. La integración continua permite detectar errores de forma temprana mediante la ejecución automática de pruebas, mientras que la entrega continua se enfoca en generar artefactos listos para su uso, como imágenes Docker, que pueden ser desplegadas posteriormente. En conjunto, este flujo mejora la calidad del software, reduce errores humanos y acelera el desarrollo.

En este proyecto, el flujo CI/CD se aplica directamente sobre el backend. Cada vez que se realiza un cambio en el repositorio, se ejecutan pruebas unitarias y de integración para validar que la lógica de negocio, la comunicación entre capas y la interacción con las bases de datos funcionen correctamente. Posteriormente, se construye una imagen Docker de la aplicación y se publica en un registry, garantizando que siempre exista una versión actualizada y lista para ser utilizada. Esto es especialmente relevante debido a la arquitectura basada en microservicios y el uso de múltiples tecnologías, lo que requiere asegurar consistencia y estabilidad en todo el sistema.

Responsabilidades

- Validar automáticamente el código ante cada cambio
- Ejecutar pruebas unitarias y de integración
- Detectar errores antes de que lleguen a producción
- Construir la aplicación de forma reproducible
- Generar imágenes Docker consistentes
- Publicar las imágenes en un registry
- Mantener versiones actualizadas del sistema
- Garantizar la estabilidad de la rama principal
- Automatizar el proceso de entrega del software
- Reducir intervención manual y errores humanos

Flujo de Datos y Uso del Balanceador

El sistema utiliza una arquitectura basada en microservicios donde todas las solicitudes ingresan inicialmente a través del balanceador de carga Nginx.

Nginx actúa como punto de entrada principal y se encarga de redirigir las solicitudes hacia el servicio correspondiente según la ruta solicitada.

Flujo General de Solicitudes

1. El cliente realiza una solicitud HTTP al sistema.
2. Nginx recibe la solicitud y analiza la ruta:
 - Las rutas `/api/` son dirigidas hacia la API principal.
 - Las rutas `/search/*` son dirigidas hacia el microservicio de búsqueda.
3. Cuando la solicitud corresponde a la API principal:
 - La API valida autenticación mediante Keycloak.
 - Se consulta Redis para verificar si existe información cacheada.
 - Si la información existe en caché, la respuesta se retorna directamente.
 - Si no existe caché, la API continua por las diferentes capas de controller y services hasta llegar a la capa de DAO donde se obtienen los datos
 - La respuesta obtenida puede almacenarse nuevamente en Redis si no lo estaba antes.
4. Cuando la solicitud corresponde al microservicio de búsqueda:
 - Las rutas del servicio reciben la solicitud y la dirigen hacia el controller correspondiente.
 - El controller valida los parámetros básicos de la búsqueda y delega la operación al service.
 - El service utiliza la capa DAO para interactuar con ElasticSearch.
 - El DAO ejecuta las consultas sobre los índices de productos almacenados en ElasticSearch.
 - Los resultados obtenidos son retornados desde el DAO hacia el service.
 - En operaciones de reindexación, el microservicio obtiene nuevamente los productos desde la API y actualiza los índices almacenados en ElasticSearch.
5. Finalmente, la respuesta regresa al cliente a través de Nginx.